CODEASURA · ENGINEERING DEEP DIVE

Rate Limiting

The complete, production-grade visual guide — from "what is it" to "how does Stripe scale it to millions of QPS." Written so a fresher gets every concept, and a staff engineer still walks away with something new.

Freshers | Senior Engineers | Architects | SRE / DevOps | Interviewers

30 SECTIONS · 30+ VISUAL DIAGRAMS · PRODUCTION CODE · INTERVIEW PREP

Think · Build · Scale

youtube.com/@CodeAsura

30 SECTIONS · 30+ VISUAL DIAGRAMS · PRODUCTION CODE · INTERVIEW PREP
30 SECTIONS · DIAGRAMS · CODE · INTERVIEW PREP

Table of Contents

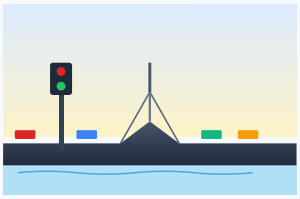
Read top to bottom for a complete mental model. Or jump to any section — each one stands alone.

01 Problem it solves
02 Problem it does not solve
03 Real-world example
04 Why we need it
05 What it is
06 What it is not
07 Prerequisites
08 How it works internally
09 Visual diagrams
10 Types & variations
11 When to use it
12 When not to use it
13 Where to use it
14 Who it is for
15 Alternatives
16 Pros
17 Cons
18 Tradeoffs
19 Implementation
20 Tools, libraries, services
21 Common mistakes
22 Failure modes
23 Observability
24 Security impact
25 Scalability impact
26 Cost impact
27 Testing strategy
28 Direct interview questions
29 Indirect interview questions
30 Final cheat-sheet

Think · Build · Scale

youtube.com/@CodeAsura

01 Problem it solves



🌉 REAL-WORLD METAPHOR

A traffic gate before a narrow bridge

The bridge can carry only so much weight. The gate decides who crosses now and who waits. Without it, the bridge collapses for *everyone* — even the careful drivers.

What pain exists without it?

Without a rate limiter, **any single client can consume an unbounded share of your server's resources** — CPU, memory, database connections, network bandwidth, third-party API quota, money. Three concrete pains follow:

Pain 1 — Resource starvation
One looping client (a buggy mobile app, a scraper, a malicious bot) pegs CPU at 100% and every other user sees timeouts. Your "good" customers pay the price for one misbehaving one.

Pain 2 — Cost explosion
Pay-per-use APIs (OpenAI, Twilio, Google Maps, AWS Lambda) charge per call. A runaway loop can burn $10,000 in 20 minutes. Without a limit, your AWS bill becomes a denial-of-wallet attack.

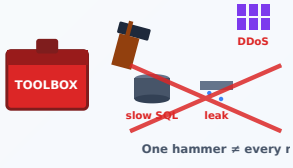
Pain 3 — Cascading failure
Service A overloads service B, which overloads database C. Connection pools fill, retries amplify load, the whole stack falls. This is how outages turn into 4-hour incidents.

Pain 4 — Abuse & fraud
Login brute force, OTP spam, coupon code enumeration, scraping, fake-account creation. Without limits, attackers try millions of guesses per minute for free.

MENTAL MODEL

A rate limiter is the **traffic light at the entrance to your system**. It does not make your system faster. It makes sure your system stays alive when traffic surges, attackers attack, and bugs ship.

02 Problem it does not solve



🔨 REAL-WORLD METAPHOR

Wrong tool for the wrong job

A hammer fixes nails, not slow databases or memory leaks. Rate limiting is one specific tool — using it to "fix" everything will break things while leaving the real problem untouched.

Where people wrongly apply it

Rate limiting is widely misunderstood. It is *not* a Swiss-Army knife. Here is what it explicitly does **not** solve:

People think it solves...	...but it actually doesn't, because
Slow endpoints	If a single request takes 5 seconds, rate limiting just means fewer slow requests. Use caching, query tuning, or async processing instead.
DDoS attacks (alone)	A volumetric DDoS hits your network bandwidth before reaching your app. You need a CDN / WAF / scrubbing service (Cloudflare, AWS Shield) at the edge. Rate limiting helps with L7 floods, not L3/L4.
Authentication / authorization	Rate limiting controls *how often* a request happens, not *whether it should*. A stolen token within budget still works.
Capacity planning	If your system can do 1k QPS and you set a limit of 1M QPS, the limit doesn't help. Rate limits are a backstop, not a replacement for autoscaling.
Bad code paths	An N+1 query is still an N+1 query when rate-limited. You're just hiding the symptom.
Fairness across tenants	A naive global limit lets the loudest tenant consume everything. You need *per-tenant* quotas or fair queuing — a different problem (see Section 10).

> **COMMON MISCONCEPTION**
>
> "We added rate limiting, so we're safe from DDoS." No. A 50 Gbps SYN flood saturates your uplink before any request reaches your application code. Rate limiting is a layer-7 tool. Layer-3/4 attacks need a different defense.

03 Real-world example

One story from production

📖 The OTP nightmare at a fintech (true pattern, names changed)

A payments app shipped a new "resend OTP" button. The button had no debounce on the client and no rate limit on the server. Within 24 hours of release:

- A user with a flaky network tapped "Resend OTP" 47 times in 3 minutes. SMS provider charged for all 47.
- A bot discovered the unprotected endpoint and started enumerating phone numbers — 40,000 SMS in one hour.
- The SMS provider auto-suspended the account after a $14,000 spike. Real users couldn't log in for 5 hours.
- Twitter incident. CTO called at 2am. Post-mortem the next morning.

The fix shipped within hours:

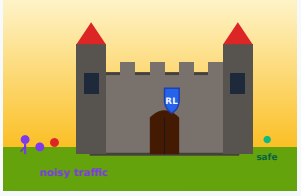
- Per-phone-number limit: `3 OTPs / 5 minutes`, max `10 / day`.
- Per-IP limit: `20 OTP requests / hour`.
- Global circuit breaker: if total OTPs/min > 10× baseline, page on-call.

Lesson: Rate limiting is not premature optimization. It's seatbelt engineering. The cost of adding it on day 1 is a few lines of code. The cost of adding it after an incident is your weekend, your reputation, and sometimes your job.

PATTERN, NOT JUST A STORY

Replace "OTP" with: password reset emails, file uploads, AI completions, search queries, payment retries, signup attempts. Same shape, same failure mode, same fix.

04 Why we need it



🏰 **REAL-WORLD METAPHOR**

A fortress wall protecting the village

The wall doesn't kill enemies — it keeps them *controlled*. Defenders inside (your DB, AI, payments) stay productive while bad actors are filtered at the gate.

Business + technical reasons

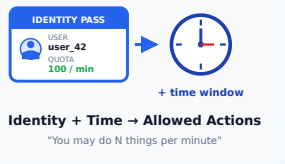
Business reasons

- **Predictable cost.** Cap third-party API spend. No surprise $80k bill.
- **Tiered monetization.** Free tier 100 req/day, Pro 10k, Enterprise unlimited. Rate limits are how you sell tiers.
- **SLA protection.** Guaranteed latency for paying customers means non-paying traffic must be capped.
- **Compliance & abuse prevention.** Required for regulated industries (banking, healthcare).
- **Brand reputation.** "Stripe never goes down" is partly because Stripe rate-limits aggressively.

Technical reasons

- **Resource protection.** Bound CPU, memory, DB connections, file descriptors.
- **Backpressure.** Tell upstream "slow down" before downstream collapses.
- **Cascading failure prevention.** Stop overload from spreading service-to-service.
- **Bot & abuse defense.** Brute force, scraping, credential stuffing — all cost-multiplied by limits.
- **Fairness.** Multi-tenant systems stay fair; one tenant can't starve others.
- **Graceful degradation.** Reject some traffic with HTTP 429 rather than crash for everyone.

05 What it is



🪪 REAL-WORLD METAPHOR

A library card with a daily borrow limit

Your ID says *who* you are. The rule says *how many books* per day. The librarian (limiter) checks both before letting you take more.

Clean definition

DEFINITION

A rate limiter is a control mechanism that restricts the number of operations a client can perform within a defined time window, by tracking usage against a quota and rejecting or delaying operations that exceed it.

Break it down:

Control mechanism
A stateful component — code, middleware, or service — that observes traffic and decides

Operations
Usually HTTP requests, but also: queue messages, DB queries, login attempts, function

allow / deny.

invocations, websocket messages.

Client
Identified by something — IP, user ID, API key, tenant ID, session, device. The "key" you bucket on.

Time window
Per second, per minute, per hour, per day. Rolling or fixed.

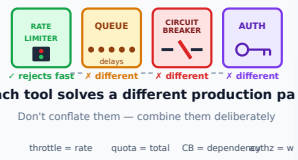
Quota
The number — e.g., 100 requests / minute. Often expressed as "rate" + "burst".

Reject or delay
Two strategies. Reject = HTTP 429 immediately. Delay = queue the request and serve when allowed (throttling).

Mathematically, a rate limiter answers one question for every incoming request:

```
// For request R from client C at time T:
function allow(C, T) {
  usage = getUsage(C, window=[T-W, T])
  return usage < quota(C)
}
```

06 What it is not



🔀 REAL-WORLD METAPHOR

Different tools, different problems

A bouncer (limiter) rejects you at the door. A queue makes you wait. A circuit breaker shuts the venue when the kitchen catches fire. An ID check (auth) decides if you're allowed at all. *All four can coexist.*

Avoid confusion with similar things

Term	What it does	Different from rate limiting because...
Throttling	Slows down requests (delays) instead of rejecting	Throttling is one *strategy* of rate limiting; rate limiting can also reject
Load shedding	Drops requests when system is overloaded	Reactive (based on system health), not proactive (based on per-client budget)
Circuit breaker	Stops calling a downstream that's failing	Protects you from *them*; rate limiting protects you from *your callers*
Backpressure	Signal to slow upstream producers (e.g., reactive streams)	A flow-control concept; rate limiting is a specific implementation pattern
Quota	Total allowed amount (e.g., 1M req/month)	Quota is a long-window limit; "rate limit" usually means short-window (sec/min)

Term	What it does	Different from rate limiting because...
Bulkhead	Isolates resource pools (e.g., separate thread pools per tenant)	Isolation by partitioning resources, not by counting requests
QoS / Traffic shaping	Prioritizes packets at network layer	L3/L4 concept; rate limiting is L7 (application)
Firewall / WAF	Filters by content / signature	Decides on *what* the request is, not *how often* it happens

WATCH FOR THIS IN INTERVIEWS

Candidates often confuse rate limiting with load shedding. Easy distinction: rate limiting is "you specifically have used too much"; load shedding is "the whole system is hot, sorry."

07 Prerequisites

REAL-WORLD METAPHOR

Building blocks before the cathedral

You can't pour the roof before the walls. HTTP semantics, Redis atomicity, identity design, and distributed-systems intuition are the foundation. Skip any one and the limiter wobbles in production.

What you must know before this

You don't need a CS degree, but these concepts make rate limiting click:

HTTP basics
Status codes (especially 429), request lifecycle, headers like `Retry-After` and `X-RateLimit-Remaining`.

Time semantics
Wall clock vs monotonic clock, time skew across servers, what a "rolling window" means.

Concurrency
Race conditions on shared counters; why `GET + INCR` is broken; atomic operations and CAS.

Distributed state
Why a single in-memory counter does not scale across N app servers; what shared state means.

Redis primitives
INCR, EXPIRE, ZADD/ZREMRANGEBYSCORE, Lua scripts, pipelining. Most production limiters live in Redis.

Probability & statistics
For approximate algorithms (sliding window approximation), and for thinking about p99 vs average load.

Caching & TTLs

Networking layers

> Why keys expire, how cache eviction interacts with limit windows.

> Where a limiter sits — L4 vs L7, reverse proxy vs API gateway vs application.

> **MINIMUM VIABLE KNOWLEDGE**
> If you understand "store a counter in Redis with a TTL, and check it before processing each request," you have enough to read the rest of this document. Everything else is refinement.

08 How it works internally

Step-by-step flow

The internals depend on the algorithm (Section 10 has all of them). Here is the logical flow common to *any* rate limiter:

1. **Identify the client.** Extract a key — IP, user ID, API key, or a composite like `tenant:user:endpoint`. The key determines whose budget to check.
2. **Look up the policy.** What is this key's quota? Free user = 100/min. Pro user = 10k/min. Different endpoints often have different policies.
3. **Check current usage.** Read the bucket/counter for this key in the relevant time window.
4. **Make the decision.**
   - If `usage < quota` → allow, increment counter, forward request.
   - If `usage ≥ quota` → reject (429) or queue (throttle).
5. **Update the state atomically.** Increment or refill the bucket. Must be atomic across replicas — race conditions here cause overcounting or under-rejecting.
6. **Set response headers.** Standard ones: `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset`, `Retry-After`. These let clients self-throttle.
7. **Emit telemetry.** Counter for allowed/rejected. Histograms for usage. Alerts when rejection rate spikes.

The interesting engineering is in steps 3–5. Specifically:

- **Where is the state?** Local memory (fast, doesn't scale) → Redis (most common) → distributed CRDT (sophisticated).
- **How do we ensure atomicity?** Lua scripts in Redis, or atomic INCR/EXPIRE pairs, or token-based optimistic concurrency.
- **How precise is the count?** Exact counts are expensive. Approximate counts (e.g., sliding-window approximation) are 99% accurate at 10× the throughput.

09 Visual diagrams

Four mental models, four pictures. Print these on your office wall.

9.1 Request flow — the happy path and the rejection path

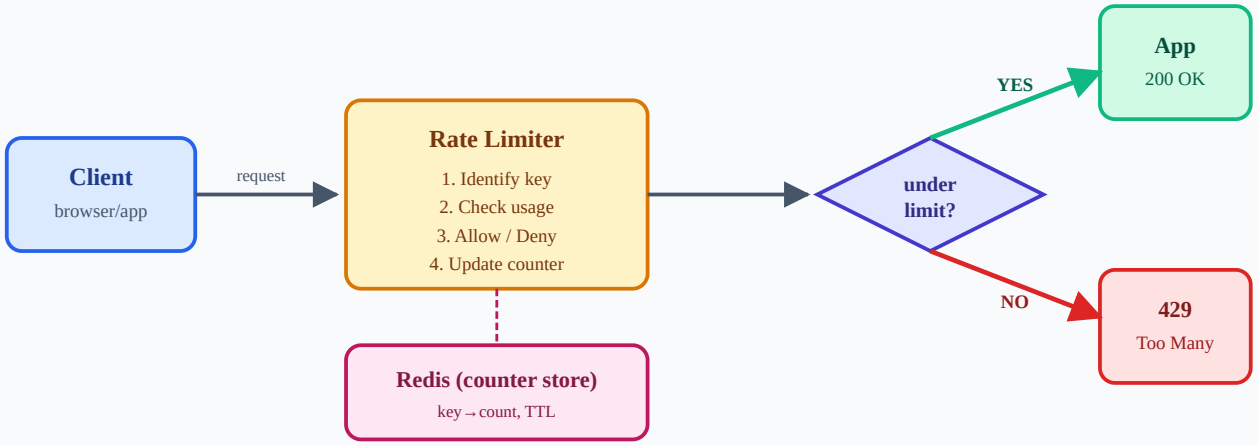


Figure 9.1 — Every request passes through the limiter, which consults a counter store and either forwards the request or returns 429.

9.2 Data flow — the token bucket in motion

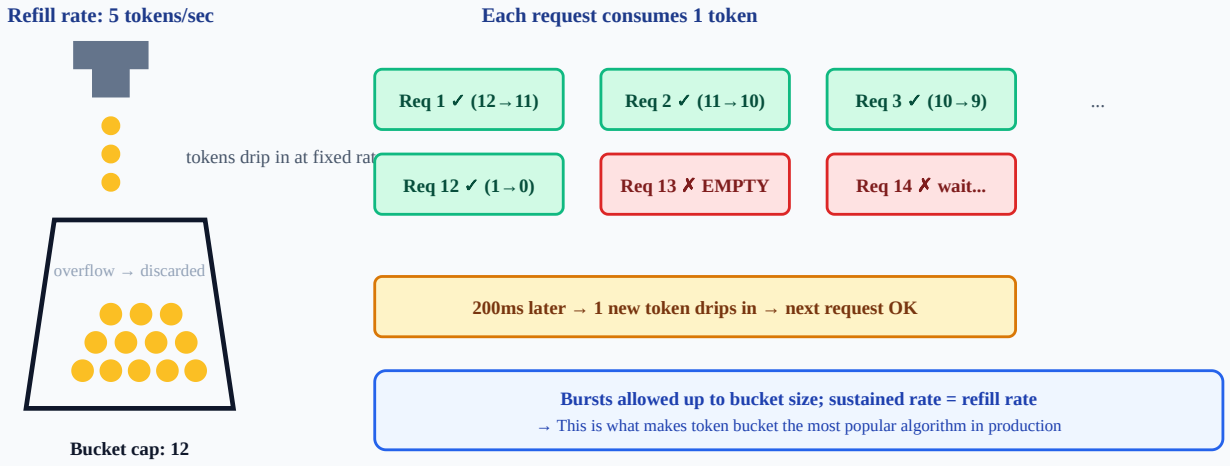


Figure 9.2 — Token bucket: the bucket fills at a fixed rate; each request takes one token; an empty bucket means rejection.

9.3 Component flow — where the limiter lives in your stack

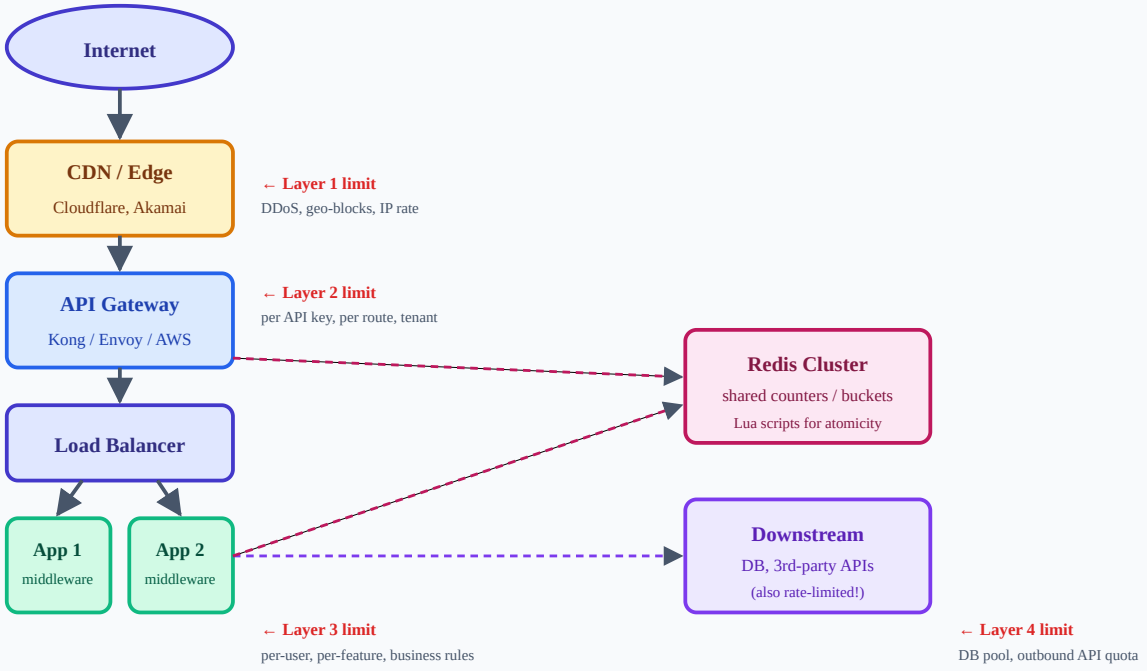


Figure 9.3 — Layered limits. Each layer catches a different attack class. Defense in depth: multiple limits, increasing precision as you go deeper.

9.4 Failure flow — what happens when things go wrong

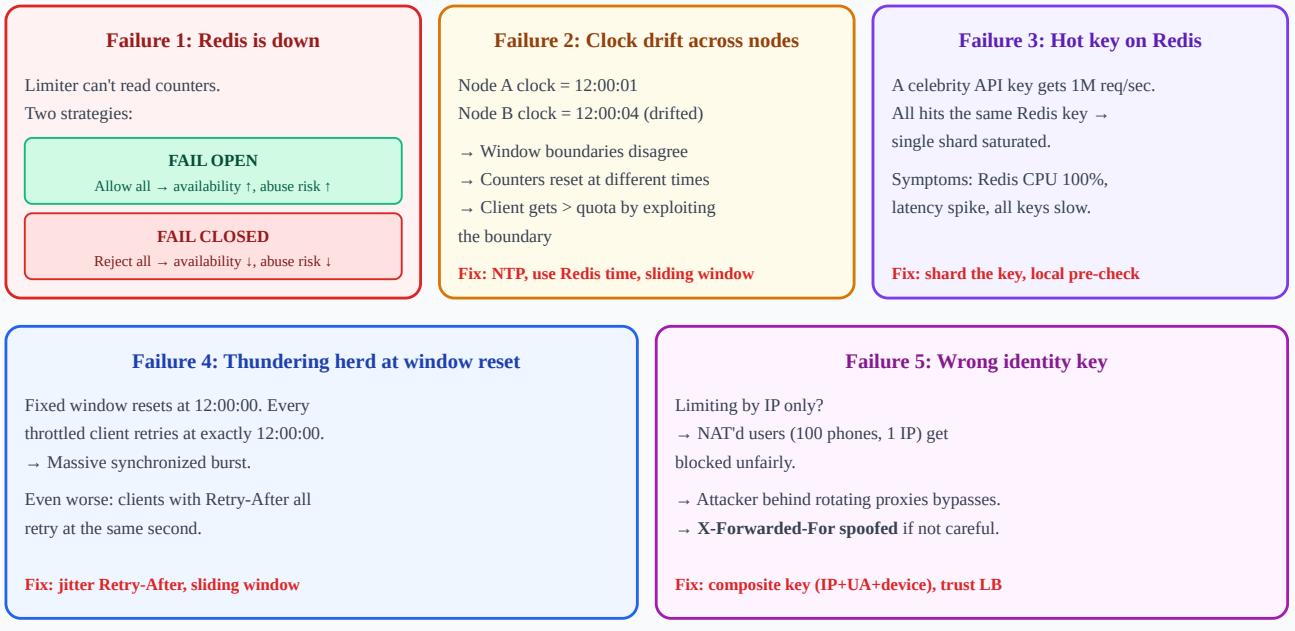


Figure 9.4 — Five failure modes you will see in production. Recognize the pattern, know the fix.

10 Types & variations

Different algorithms — pick the right tool for the job

Five algorithms dominate production. Understand all five; you will be asked about them.

10.1 Fixed Window Counter

Divide time into fixed windows (e.g., 1 minute). Count requests in the current window. Reset on boundary.

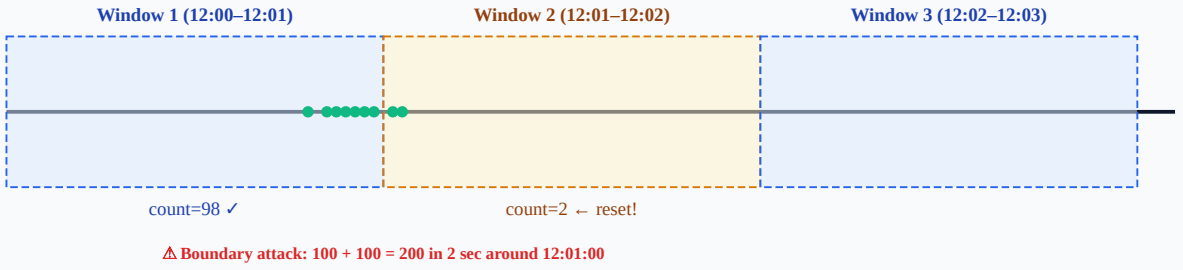


Figure 10.1 — Fixed window's flaw: a client can fire 100 at 12:00:59 and another 100 at 12:01:00 — 200 requests in <1 second, technically within "100/min".

Pro Trivial to implement (one counter + TTL). **Con** Boundary burst (2× the limit possible). **Use** when precision is not critical and you need O(1) memory.

10.2 Sliding Window Log

Store a timestamp for every request in a sorted set. Count timestamps in `[now - W, now]`. Drop expired ones.

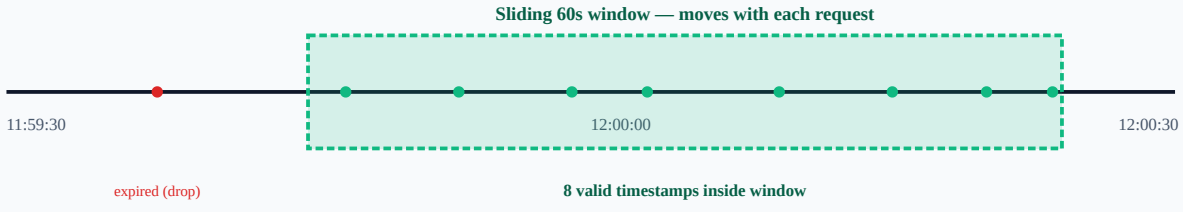


Figure 10.2 — Sliding window log keeps an exact list of recent request times. Most accurate, but O(N) memory.

Pro Exact, no boundary burst. **Con** O(N) memory per key — a heavy user with 1M req/min costs 1M timestamps. **Use** for low-traffic, high-precision needs (e.g., login attempts).

10.3 Sliding Window Counter (approximation)

Maintain two fixed-window counters (current and previous). Estimate sliding count by weighting:

```
count ≈ current_count + previous_count × (overlap_fraction_with_previous_window)
```

Example: 75% into the current window → `count ≈ current + 0.25 × previous`. Close to exact, but O(1) memory.

Pro O(1) memory, near-exact (<1% error in practice). **Pro** No boundary burst. **Use** Cloudflare's choice — best general-purpose algorithm.

10.4 Token Bucket

Bucket of capacity B fills at rate R tokens/sec. Each request consumes 1 token. Empty bucket = reject. (Diagram in 9.2.)

Pro Allows bursts up to B; smooth long-term rate at R. **Pro** O(1) state: just `(tokens, last_refill_ts)`. **Use** Most popular. AWS, Stripe, Google all use this. Default choice unless you have a reason otherwise.

10.5 Leaky Bucket (queue-based)

Requests enter a FIFO queue. The queue drains at a constant rate. Full queue = reject. Used to *shape* traffic into a smooth output stream.

Token bucket vs Leaky bucket

Token bucket: input = bursty, output = bursty (up to bucket). Cap on max usage.

Leaky bucket: input = bursty, output = smooth. Cap on output rate (regardless of input).

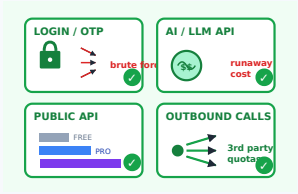
Picking between them

Use **token bucket** when you want to *allow* bursts (most APIs).

Use **leaky bucket** when downstream is fragile and needs constant pacing (legacy systems, SMS gateways, write-heavy DBs).

Algorithm	Memory	Burst	Accuracy	Best for
Fixed Window	O(1)	2× possible	Low	Quick & dirty, low-stakes endpoints
Sliding Window Log	O(N)	None	Exact	Logins, security-sensitive, low-volume
Sliding Window Counter	O(1)	None	≈99%	General-purpose web APIs
Token Bucket	O(1)	Up to bucket size	Exact	APIs that need to allow bursts (default)
Leaky Bucket	O(queue)	None (smoothed)	Exact	Traffic shaping into fragile downstream

11 When to use it



✅ **REAL-WORLD METAPHOR**

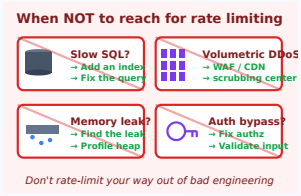
The "obvious yes" zones

Anywhere a single noisy actor can ruin everyone's day, or where one request costs real money — login, AI, public APIs with plans, outbound to paid third parties — that's where rate limiting earns its keep.

Correct use cases

- **Public APIs** — anything exposed to the internet. Always.
- **Authentication endpoints** — login, signup, OTP, password reset. Per-account + per-IP.
- **Expensive operations** — AI completions, video transcoding, report generation, search.
- **Pay-per-use downstream** — calls to Twilio, OpenAI, Google Maps, Stripe. Limit before you call.
- **Multi-tenant SaaS** — per-tenant fairness, plan-tier enforcement.
- **Webhook receivers** — third parties may misfire and flood you.
- **Write-heavy DB endpoints** — anything that creates rows or sends emails.
- **Internal services** — yes, internal too. A bad deploy in service A shouldn't kill service B.

12 When not to use it



⛔ **REAL-WORLD METAPHOR**

Painkillers don't fix broken bones

Rate limiting masks the symptom of an overloaded system. If the underlying cause is a slow query or a memory leak, you've just made the bug invisible — until it returns at the worst possible moment.

Wrong use cases

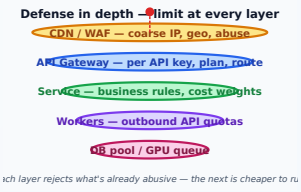
- **Health-check endpoints** — load balancers depend on these. Limiting them causes false-positive node removals.
- **Static asset delivery** — let the CDN handle it. Limiting `/css/main.css` per IP just breaks legit users.
- **Read-only, cache-friendly endpoints** with cheap responses — caching solves the problem more elegantly.
- **Real-time bidding / latency-critical** systems where the latency added by a Redis call is unacceptable. Use local in-memory limits with eventual sync.

- **Internal jobs you control** — if you wrote both producer and consumer, fix the producer instead.
- **To replace capacity planning** — limits don't add capacity; they only prevent overload.

ANTI-PATTERN

Setting a global limit so high it never triggers in practice ("just in case") gives a false sense of security. Set limits where they will *actually fire* under abuse, and tune them with real traffic data.

13 Where to use it



REAL-WORLD METAPHOR

A multi-layer security cake

Place limits at every layer of the stack — like a sieve cascade. Coarse filters at the top (CDN), precise filters near the bottom (per-feature business rules). One layer alone is never enough.

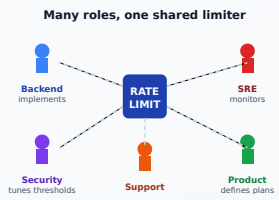
System layer placement

Rate limiting is not a single-layer thing. Mature systems run limits at multiple layers, each catching a different attack class.

CDN / Edge	**Cloudflare, Akamai, Fastly.** Geographic, IP-based, bot-score. Catches volumetric attacks before they enter your network. Cheapest tier.
WAF	**AWS WAF, Cloudflare WAF.** Pattern-based + simple rate rules. Good for "block any IP doing > 100 req/min".
API Gateway	**Kong, Tyk, Apigee, AWS API Gateway, Envoy.** Per-API-key, per-route, per-tenant. Centralized policy. Most production limits live here.
Reverse Proxy	**Nginx, HAProxy, Envoy.** Coarse-grained, very fast. `limit_req_zone` in nginx is the classic.
Service Mesh	**Istio, Linkerd.** Service-to-service limits to prevent cascading failures inside the mesh.
Application	**Middleware in your code.** Business-aware limits: per-user, per-feature, per-plan-tier. Where complex logic lives.
Database	**Connection pools (HikariCP, PgBouncer).** Implicit rate limit via finite connections. Plus per-statement timeouts.

Queue / Worker	**Kafka, SQS, Celery.** Concurrency limits and rate limits on consumers. "Process at most 10 jobs/sec".
Outbound	**Client-side limiter on calls to 3rd-party APIs.** Stay under their quotas; don't get your account banned.

14 Who it is for



👥 REAL-WORLD METAPHOR

A shared resource, many stakeholders

Like a shared kitchen in an office — engineers cook the limits, SRE keeps the appliances running, security checks the locks, product writes the rules. Confusion happens when nobody owns the policy.

Roles & concerns

Backend Engineer
Implements middleware, picks algorithm, integrates with Redis, handles 429 responses correctly.

Architect
Decides where limits live in the stack; sets limits per tier; designs for fairness and scale.

SRE / DevOps
Operates Redis cluster, sets alerts, builds dashboards, runs load tests, manages incidents.

Security Engineer
Tunes limits for credential stuffing, OTP abuse, scraping; integrates with WAF and bot detection.

Platform / API Team
Provides rate-limiting as a self-service primitive for product teams. Owns gateway config.

Product Manager
Defines tier-based quotas tied to monetization (free 100/day, pro 10k/day).

AI Engineer
Critical for LLM endpoints — token-based limits, cost ceilings per user, model-tier routing.

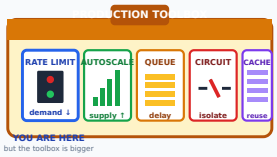
Frontend / Mobile Eng

Reads `X-RateLimit-*` headers, implements client-side throttling and exponential backoff.

Data Engineer

Throttles outbound API pulls; rate-limits ingestion to avoid overwhelming downstream stores.

15 Alternatives



🧰 **REAL-WORLD METAPHOR**

Pick the right tool from the box

Rate limiting reduces demand. Autoscaling adds supply. A queue delays. A circuit breaker isolates. A cache reuses. The best production answer is rarely one tool — it's the right combination.

Other ways to solve the same or related problem

Alternative	What it does	When better than rate limiting
Caching	Serve cached responses	Read-heavy endpoints with stable data — cheaper than rejecting
Autoscaling	Add capacity to meet demand	Legitimate traffic surges; you have budget
Load shedding	Drop requests when overloaded	System-health-driven, not per-client; complementary
Circuit breaker	Stop calling failing downstream	Different problem (downstream failure protection)
Bulkheads	Isolate resource pools per tenant	Strong isolation needs; expensive to operate
Queue + async	Buffer the work and process at your pace	Workload doesn't need synchronous response
CAPTCHA / proof-of-work	Make each request expensive for client	Bot defense at signup / login
WAF rules	Block patterns & signatures	Known-bad traffic; not volume-based
Quotas (long-window)	Daily/monthly total caps	Billing & abuse over long periods, not real-time bursts

In practice, you stack these. A real production system has all of: caching + autoscaling + rate limiting + circuit breakers + load shedding. Rate limiting is one tool, not the only one.

16 Pros

🏆 **REAL-WORLD METAPHOR**

The wins on the trophy shelf

A good limiter wins quietly: smooth latency under bursts, fair sharing across tenants, lower cloud bills, fewer 4am pages. You only notice it when it's missing.

Benefits

- **Predictable resource use.** Bound your worst case. Capacity planning becomes possible.
- **Abuse prevention.** Brute force becomes economically infeasible for attackers.
- **Cost control.** Pay-per-use APIs and infra have a hard cap.
- **Fairness.** One noisy neighbor can't starve the others.
- **Graceful degradation.** 429 to some > outage for everyone.
- **Tier monetization.** Sell quotas as a product feature.
- **SLA enforcement.** Reserve capacity for paying customers.
- **Compliance.** Meets requirements in regulated industries (PCI, HIPAA, banking).
- **Cheap to add.** Rate limiting middleware is dozens of lines. ROI is enormous.

17 Cons

⚠️ **REAL-WORLD METAPHOR**

The warning signs to read

Every defense has a downside. Wrong key punishes innocent users. Strict thresholds kill conversions. Redis becomes a new SPOF. The job isn't to avoid limits — it's to know what you're trading.

Weaknesses

- **Adds latency.** Every request now does a Redis round-trip (~1ms typically, but it's a real cost).
- **New dependency.** Redis (or whatever store) is now in the critical path. If it fails, you must decide fail-open vs fail-closed.
- **Hard to set right.** Too low = rejects legit users. Too high = no protection. Tuning is iterative.
- **Identity is messy.** IPs change, behind NAT they're shared, behind proxies they can be spoofed. Picking the right key is tricky.
- **Distributed consistency overhead.** Atomicity across replicas requires Lua or CAS, which adds complexity.

- **Race conditions.** Naive implementations let bursts slip through (compare-then-set without atomicity).
- **UX problem.** 429 feels like a bug to users. You need good error messaging and client SDKs that retry sensibly.
- **Hot keys.** A single popular API key can saturate one Redis shard.

18 Tradeoffs

⚖️ **REAL-WORLD METAPHOR**

Every choice has a price tag

Sliding log is exact but heavy. Local memory is fast but inconsistent. Fail-closed is safe but risks an outage. Engineering is the art of choosing which knob to turn — and knowing what breaks when you do.

What you gain vs what you lose

You gain	...by giving up
Protection against abuse	Some legitimate-but-aggressive users get 429s
Predictable load	Maximum-throughput potential during spikes
Lower infra cost	Some user complaints / churn from blocked users
Multi-tenant fairness	Single-tenant max throughput
Defense in depth	Latency (Redis lookup) + complexity (new dep) + ops cost
Exact accuracy (sliding log)	Memory (O(N))
O(1) memory (token bucket)	Tunability — bucket size and refill rate must match real patterns
Burst tolerance	Predictable smoothness of downstream load
Distributed correctness	Throughput (atomic ops are slower than non-atomic)

THE CLASSIC ENGINEERING TRADEOFF

Choose your axis: **accuracy** (exact counts), **memory** (O(1) state), **throughput** (no Redis hop), **latency** (sub-millisecond). You can't max all four. Most teams pick "approximate sliding window in Redis" — sacrificing 1% accuracy and 1ms latency for a great balance.

19 Implementation

19.1 Basic — in-memory token bucket (Python)

Single-server, no dependencies. Good for understanding the algorithm. **Not for production.**

```python
import time, threading

class TokenBucket:
    def __init__(self, rate, capacity):
        self.rate = rate              # tokens per second
        self.capacity = capacity      # max tokens
        self.tokens = capacity        # start full
        self.last = time.monotonic()  # monotonic clock — never goes backward
        self.lock = threading.Lock()

    def allow(self, n=1):
        with self.lock:
            now = time.monotonic()
            # Refill based on elapsed time, capped at capacity
            self.tokens = min(self.capacity, self.tokens + (now - self.last) * self.rate)
            self.last = now
            if self.tokens >= n:
                self.tokens -= n
                return True
            return False

# Usage: 10 req/sec sustained, bursts up to 20
bucket = TokenBucket(rate=10, capacity=20)
if bucket.allow():
    handle_request()
else:
    return 429
```

Why this isn't production-ready: in-memory state means each app server has its own bucket. With N servers, the effective limit is N×rate. That's fine if you accept the looseness — but most teams want a shared, distributed limiter.

19.2 Production — distributed token bucket with Redis + Lua

The Lua script runs atomically inside Redis, eliminating race conditions across N application replicas. This is the pattern used by Stripe, Cloudflare, and most large APIs.

```lua
-- token_bucket.lua  — runs atomically in Redis
-- KEYS[1] = bucket key (e.g., "rl:user:42:api")
-- ARGV: rate, capacity, now_ms, requested_tokens

local key      = KEYS[1]
local rate     = tonumber(ARGV[1])    -- tokens / sec
local capacity = tonumber(ARGV[2])
local now      = tonumber(ARGV[3])    -- ms
local req      = tonumber(ARGV[4])

local data = redis.call('HMGET', key, 'tokens', 'ts')
local tokens = tonumber(data[1]) or capacity
local ts     = tonumber(data[2]) or now

-- Refill
local elapsed_s = (now - ts) / 1000.0
tokens = math.min(capacity, tokens + elapsed_s * rate)

local allowed = 0
if tokens >= req then
  tokens = tokens - req
  allowed = 1
end

redis.call('HMSET', key, 'tokens', tokens, 'ts', now)
-- Auto-cleanup unused keys (capacity/rate seconds = full refill time × 2)
redis.call('EXPIRE', key, math.ceil(capacity / rate * 2))

return {allowed, tokens}
```

Calling it from your application (Node.js example):

```
const Redis = require('ioredis');
const redis = new Redis({ host: 'redis-cluster', enableAutoPipelining: true });

// Load script once at startup → SHA cached for fast EVALSHA
const luaScript = fs.readFileSync('token_bucket.lua', 'utf8');
const sha = await redis.script('LOAD', luaScript);

async function checkLimit(userId, rate, capacity, cost = 1) {
  const key = `rl:user:${userId}`;
  try {
    const [allowed, tokensLeft] = await redis.evalsha(
      sha, 1, key, rate, capacity, Date.now(), cost
    );
    return { allowed: allowed === 1, remaining: Math.floor(tokensLeft) };
  } catch (err) {
    // Redis down — fail open with logging + metric
    metrics.inc('ratelimit.fail_open');
    log.error({ err }, 'limiter unavailable, allowing');
    return { allowed: true, remaining: null, degraded: true };
  }
}

// Express middleware
function rateLimit({ rate, capacity }) {
  return async (req, res, next) => {
    const key = req.user?.id ?? req.ip;
    const { allowed, remaining } = await checkLimit(key, rate, capacity);
    res.set('X-RateLimit-Limit', capacity);
    res.set('X-RateLimit-Remaining', remaining ?? 'unknown');
    if (!allowed) {
      res.set('Retry-After', Math.ceil(1/rate) + jitter());
      return res.status(429).json({ error: 'rate_limited' });
    }
    next();
  };
}
```

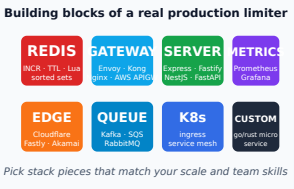
19.3 Production-grade checklist

- **Composite keys**: `rl:{tenant}:{user}:{endpoint}` — different limits per dimension.
- **Use EVALSHA, not EVAL**: cache the script SHA. Saves bandwidth on every call.
- **Pipeline / batch**: if you check multiple limits per request, pipeline them.
- **Set EXPIRE**: prevents memory leaks from inactive keys.
- **Use Redis time**: `redis.call('TIME')` avoids client clock skew across nodes.
- **Fail-open with alerting**: when Redis is unreachable, allow traffic but page the on-call.
- **Local L1 cache**: cache "definitely-rejected" decisions for 1 second to absorb flood without hitting Redis.
- **Add jitter to Retry-After**: prevents synchronized retries (thundering herd).
- **Standard headers**: `X-RateLimit-Limit`, `-Remaining`, `-Reset`, `Retry-After`.
- **Cost-weighted requests**: a search might cost 5 tokens; an LLM call might cost 100.

PRO TIP FROM PRODUCTION

Wrap the Lua call with a *circuit breaker*. If Redis is consistently slow, stop calling it for 30 seconds and serve from a degraded local-only limiter. This avoids the limiter itself becoming an outage source.

20 Tools, libraries, services



🧩 **REAL-WORLD METAPHOR**

Off-the-shelf parts that snap together

You don't build a limiter from scratch — you assemble one from Redis (state), a gateway (enforcement), middleware (business rules), and Prometheus (visibility). Each part is mature; the skill is wiring them right.

The ecosystem

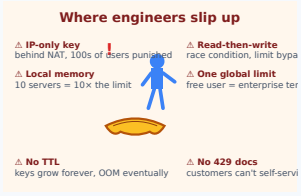
Layer	Tool	Strength
CDN/Edge	Cloudflare Rate Limiting	Edge-scale; per-IP, per-country, bot-aware
CDN/Edge	AWS WAF Rate Rules	Native to AWS stack; integrates with CloudFront
CDN/Edge	Fastly VCL	Programmable edge limits
API Gateway	Kong (rate-limiting plugin)	Open-source, Lua-based, Redis-backed; battle-tested
API Gateway	Envoy (rate_limit filter)	Service mesh standard; gRPC service for limits
API Gateway	Tyk, Apigee, AWS API Gateway	Managed gateway products with built-in limits
Reverse Proxy	Nginx (limit_req_zone)	Coarse, fast, on-box; classic for static cases
Reverse Proxy	HAProxy (stick-tables)	L4/L7 limits with shared tables across instances
Reverse Proxy	Traefik	Modern, easy K8s integration
Storage	Redis (single + cluster)	De facto standard for distributed counters
Storage	Memcached	Faster but no Lua; harder to do atomically
Storage	DynamoDB	For massive scale or AWS-native; conditional writes
Library (Java)	Resilience4j RateLimiter, Bucket4j	Bucket4j is the most flexible; Resilience4j integrates with Spring

Layer	Tool	Strength
Library (Go)	golang.org/x/time/rate, redis_rate	Standard token bucket; redis_rate for distributed
Library (Node.js)	express-rate-limit, rate-limiter-flexible	rate-limiter-flexible supports many backends and algorithms
Library (Python)	slowapi (FastAPI), django-ratelimit, limits	"limits" is backend-agnostic and clean
Library (Ruby)	rack-attack	Standard for Rails; powerful DSL
Service Mesh	Istio (envoy-based)	Service-to-service limits via mesh policies
Specialized	Stripe's open-source limiter	Token bucket + concurrency; tested at scale
SaaS	Upstash Ratelimit, Unkey	Hosted; great for serverless / edge

HOW TO CHOOSE

Just starting? Add `express-rate-limit` or `slowapi` with a Redis backend. **API gateway?** Kong or Envoy. **Cloud-native?** Use your provider's WAF + your gateway's plugin. **Edge / serverless?** Upstash Ratelimit or Cloudflare. **High scale?** Roll your own Lua scripts on a Redis cluster.

21 Common mistakes



🍌 **REAL-WORLD METAPHOR**

The classic banana peels

The bug isn't usually in the algorithm — it's in the key. IP-only behind a NAT punishes a whole company. Local memory across 10 servers leaks 10× the limit. The interesting failures are boring engineering details.

Real mistakes developers make

1. **Limiting only by IP.** NAT'd users (corporate networks, mobile carriers) share IPs. Authenticated users should be limited by user ID; anonymous by IP+user-agent+device fingerprint.
2. **Trusting `X-Forwarded-For` directly.** Spoofable. Always read the rightmost trusted-proxy hop, configured explicitly.
3. **Non-atomic increments.** `GET key; INCR if < limit` is a race. Use Lua or atomic Redis ops.
4. **No EXPIRE on the key.** Memory leaks. Inactive keys stay forever.
5. **One global limit for all endpoints.** A search and a login have different cost profiles; they need different limits.

6. **Setting limits without baseline data.** "100/min sounds reasonable" — based on what? Measure first.
7. **Same limit for free and paid users.** No differentiation = no monetization.
8. **Returning 500 instead of 429.** 429 is the standard. Clients (and CDNs) understand it; 500 confuses everyone.
9. **No `Retry-After` header.** Clients retry immediately, get 429 again, repeat. Useless feedback loop.
10. **No jitter in retries.** Synchronized retries cause thundering herds.
11. **Limiting health checks.** Load balancers get 429, mark node unhealthy, take it out of rotation. Outage.
12. **Hard-coded limits.** Make them configurable at runtime so you can tune without redeploy.
13. **No metrics.** If you can't see rejection rates, you can't tune limits or detect attacks.
14. **Limiting before authentication.** If the auth check is cheap, do it first. Limit by user ID, not IP, where possible.
15. **Forgetting outbound limits.** Limiting incoming but not your calls to OpenAI is how you get banned.

22 Failure modes

How it breaks in production

Beyond the five in Figure 9.4, here are the classics:

- **Limiter latency dominates request latency.** Redis goes 99th-percentile slow → every request slows. Use timeouts on the limiter call (e.g., 50ms) and fail-open if exceeded.
- **Hot key.** One Redis key (a popular API key, or a global limit) gets millions of ops/sec to a single shard. Mitigation: shard the key into N sub-keys and route via hash; reconcile on read.
- **Counter inflation.** A bug increments multiple times per request. Real users get 429 mysteriously. Hard to debug — instrument the increment path.
- **Counter never resets.** Forgot EXPIRE; counter from yesterday still counts.
- **Mismatched windows across nodes.** Each node uses its local clock. NTP drift means windows misalign. Use a single time source (Redis time).
- **Thundering herd at reset.** Fixed-window all clients retry at second 0. Add jitter, or use sliding window.
- **Limiter killed by its own attacker.** Attacker rotates IPs at the rate of new-key creation. Each new IP creates a new Redis key. Memory blows up. Mitigation: bound key creation per parent (e.g., per /24 subnet).
- **Fail-closed cascade.** Redis blip → limiter rejects all → CDN/LB sees mass 429 → marks origin unhealthy → outage. Default to fail-open with cap-and-alert.
- **Configuration drift.** Different gateway pods load different limit configs. Symptoms: same user randomly rejected. Mitigation: centralized config + consistency check.

- **Plan-tier downgrade race.** User downgrades from Pro to Free; in-flight requests use stale Pro quota. Tolerable, but be aware.

23 Observability

Logs, metrics, alerts, dashboards

Metrics (RED + USE)

- `ratelimit.allowed.count` — counter, allowed requests
- `ratelimit.rejected.count` — counter, rejected (429s)
- `ratelimit.rejected.ratio` — derived: rejected / total
- `ratelimit.check.latency` — histogram (p50, p95, p99)
- `ratelimit.tokens.remaining` — gauge per top-N keys
- `ratelimit.fail_open.count` — counter for Redis-down events
- `redis.command.latency{cmd=evalsha}` — Redis-side health
- `ratelimit.unique_keys` — gauge; sudden growth = key cardinality attack

Logs

Don't log every allowed request — too noisy. Log every rejection at INFO with: key, endpoint, current usage, limit, IP, user agent. This is your forensic trail when investigating incidents.

Alerts

- **Page**: rejection ratio > 5% sustained for 5 minutes. Means either an attack or your limits are too tight.
- **Page**: Redis fail-open count > 0 for > 60 seconds. Limiter is degraded.
- **Page**: limiter latency p99 > 50ms. Limiter is hurting users.
- **Ticket**: unique-keys grows > 10× baseline. Key cardinality attack.
- **Ticket**: a single key sustained at 100% of its limit for an hour. Likely abuse or stuck client.

Dashboard layout (top to bottom)

1. Top row: total RPS, allowed RPS, rejected RPS, rejection %.
2. Middle: rejection rate by endpoint, by user tier, by region.
3. Below: top 10 rejected keys (table). Top 10 nearly-saturated keys.
4. Bottom: limiter latency p50/p95/p99, Redis health, fail-open events.

> **INSIGHT**
>
> Treat the rejection rate as a *signal*. A flat near-zero rate means you set limits too high. A consistent 1–3% rate means you're catching abusers and your limits are well-tuned. A sudden spike is either a new attack pattern or a legitimate traffic surge — investigate immediately.

24 Security impact

🛡️ **REAL-WORLD METAPHOR**

One shield among many

Rate limits stop a single attacker hammering one endpoint. They don't stop a botnet rotating IPs, a stolen credential, or a logic flaw. Defense in depth: limit + WAF + risk scoring + auth + fraud system.

What it helps and doesn't

Helps with

- Brute-force login attacks
- Credential stuffing
- OTP / 2FA SMS abuse
- Coupon code enumeration
- Account-creation flooding
- Scraping & web harvesting
- Layer-7 DDoS (HTTP floods)
- Denial-of-wallet (cost amplification)
- API abuse by stolen keys

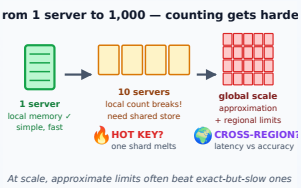
Does NOT help with

- L3/L4 volumetric DDoS (need scrubbing)
- Slow-rate attacks (Slowloris) — under threshold
- Credential theft itself
- Authorization bugs
- Injection vulnerabilities (SQLi, XSS)
- Insider threats with valid quotas
- Supply-chain compromises
- Phishing

CAREFUL

Rate limiting on login endpoints can *create* a denial-of-service vector: attacker triggers your limit on a legitimate user's account, locking them out. Mitigations: limit by IP+user combo (not just user), use CAPTCHA after threshold, allow administrative bypass.

25 Scalability impact



📈 **REAL-WORLD METAPHOR**

A growing city needs a shared census

One person counts a small village from memory. A thousand counters in a thousand villages produce nonsense unless they synchronize. Limits at scale demand shared state, accept approximation, and watch for hot keys.

Single server vs distributed

Single server
In-memory bucket. Fast (no network), simple, but limit is per-process. With N replicas, real limit ≈ N × configured. Ok for "best effort".

Multiple servers, shared Redis
All replicas talk to one Redis. Each request = +1 RTT (~0.5ms). Redis is now in the critical path; needs HA.

Hot-key sharding
A celebrity user's key gets too hot. Shard the key into K sub-keys (`rl:user:42:shard:0..K-1`). Read/sum on check. Trades exactness for distribution.

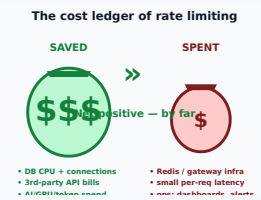
Local + remote two-tier
Local cache absorbs floods; sync to Redis every N ms. Massive throughput, slight inaccuracy.

Probabilistic data structures
Count-Min Sketch instead of counters → fixed memory, approximate counts. Good for "did this client exceed the limit?" at huge cardinality.

Edge limiters
Cloudflare/Fastly do the limit at the edge POP. Each POP has its own state; eventually consistent. Massive scale, slightly fuzzy globally.

Real-world numbers (rough): a single Redis instance can do ~100k limiter checks/sec. Cluster: 1M+ /sec. For more, you shard, go local-first, or move to edge.

26 Cost impact



💰 REAL-WORLD METAPHOR

A small premium that prevents big bills

Like insurance — you pay a little every month so a runaway loop doesn't generate a $20k bill at 3am. Cost-weighted limits (not just request count) capture the real spend driver.

Where the bill goes

Cost	Magnitude	Notes
Redis infrastructure	Low–Med	~$50–500/month for an HA pair; cluster scales linearly
Latency added per request	Low	~0.5–2ms; affects user p99 slightly
Engineering time to build	Med	1–4 weeks initial + ongoing tuning
Engineering time to debug abuse	Med	Forensic work when limits trigger
Customer support tickets	Low–Med	"Why am I getting 429?" — needs good error messages

Cost	Magnitude	Notes
Lost revenue from rejected legit users	Variable	Tune limits to minimize false positives
Saved infra (downstream protection)	High (negative)	Often saves more than it costs by capping autoscaling
Saved 3rd-party API spend	High (negative)	One bug-induced loop without limits = $1000s/hour saved
Saved incident hours	High (negative)	Outages are expensive; limits prevent many

Bottom line: rate limiting is one of the highest-ROI infrastructure investments. The net cost is almost always negative — it pays for itself many times over.

27 Testing strategy

Unit, integration, load, chaos

Unit tests

- Bucket starts full at capacity.
- N consecutive requests at rate R succeed if N ≤ capacity.
- (N+1)th immediate request fails.
- After waiting (1/R) seconds, one more request succeeds.
- Refill doesn't exceed capacity (no over-fill).
- Refill works correctly at fractional second granularity.
- Different keys have independent buckets.
- Cost-weighted requests deduct correctly.
- Clock going backward doesn't break counts (use monotonic).

Integration tests

- End-to-end: send N+1 requests through real Express + Redis stack; assert 1 gets 429.
- Headers: `X-RateLimit-Limit`, `-Remaining`, `Retry-After` are present and correct.
- Multiple replicas: same client across two app servers shares the limit.
- Redis flake: kill Redis mid-test, verify fail-open behavior + log/metric emission.

Load tests

- Use k6, Locust, or Gatling. Send a known burst — verify rejection count matches expectation.
- Sustain at 1.5× the limit; rejection ratio should stabilize at ~33%.
- Measure limiter latency under peak load (ideally < 5ms p99).
- Test with a single hot key vs spread across many keys; observe Redis CPU.

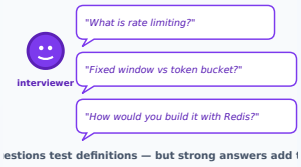
Chaos tests

- Kill Redis primary mid-traffic. Verify failover < 30s; fail-open during gap.
- Introduce 200ms Redis latency (tc netem). Verify limiter timeout fires correctly.
- Time skew test: jump app clock forward 1 hour. Verify counters don't break catastrophically.
- Clock skew test: introduce 5-second clock difference between two app nodes. Verify behavior.

Edge cases worth testing

- Negative tokens (bug-induced) — should clamp to 0.
- Capacity = 0 — should reject everything.
- Rate = 0 — defines a one-shot limit (only initial bucket).
- Very small windows (1ms) — granularity errors.
- Very large bursts after long idle — refill should cap at capacity, not overflow.

28 Interview questions — direct (named)



🎤 REAL-WORLD METAPHOR

The questions you'll be asked by name

When the interviewer says "rate limiter," they're testing the basics. The win comes from *not stopping there* — add tradeoffs, failure modes, and observability to every answer.

The interviewer says the words "rate limiting" or "rate limiter" up front.

Q1. What is rate limiting and why do we need it?

A control mechanism that caps the number of operations a client can perform in a time window. We need it for resource protection, abuse prevention, fair multi-tenancy, cost control, and graceful degradation. (Sections 1–5.)

Q2. Walk me through the major rate-limiting algorithms.

Fixed window (simple, allows boundary bursts), sliding window log (exact, O(N) memory), sliding window counter (≈99% accurate, O(1)), token bucket (allows controlled bursts), leaky bucket (smooths output). Pick token bucket as default. (Section 10.)

Q3. How do you implement a distributed rate limiter?

Shared store (typically Redis), atomic update via Lua script or atomic primitives, composite key per client/endpoint, EXPIRE for cleanup, fail-open with metrics on store failure. Walk through the Lua script in Section 19.2.

Q4. What are the tradeoffs between token bucket and leaky bucket?

Token bucket allows bursts up to bucket size while smoothing the long-term rate — great for APIs that have spiky-but-bounded usage. Leaky bucket forces a constant output rate regardless of input — great for protecting fragile downstream like SMS gateways or legacy DBs. Token bucket is more popular because real workloads are bursty. (Section 10.5.)

Q5. How do you decide what key to limit on?

Authenticated traffic: user ID or API key. Anonymous: IP + user agent + device fingerprint. For sensitive ops (login): combine — by-IP and by-account, with the more restrictive winning. Avoid IP-only because of NAT and proxies. (Section 21, mistake #1–2.)

Q6. What headers should a rate-limited API return?

`X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset` (or the IETF draft `RateLimit-*`), and `Retry-After` on 429 responses. The status code is 429 Too Many Requests. (Section 19.2.)

Q7. How do you scale a rate limiter to millions of QPS?

Three techniques: (1) shard hot keys across Redis nodes; (2) two-tier — local limiter absorbs floods, syncs periodically; (3) push to the edge (CDN). Accept some inaccuracy for throughput. (Section 25.)

Q8. What happens if Redis is down?

Fail-open by default — allow traffic, emit metric, page on-call. Fail-closed only for security-critical paths (login, payment) where availability is less important than abuse prevention. Use a circuit breaker around the Redis call so a sick Redis doesn't tank latency. (Section 22, Figure 9.4 #1.)

Q9. How is rate limiting different from load shedding?

Rate limiting is per-client, proactive ("you specifically have used too much"). Load shedding is system-wide, reactive ("the whole system is hot"). They complement each other — use both. (Section 6.)

Q10. Walk through a race condition in a naive rate limiter.

Pseudocode: `if (GET counter) < limit: INCR counter`. Two concurrent requests both read 99 (limit 100), both increment to 100 and 101 — limit exceeded. Fix: atomic INCR-then-check, or wrap in a Lua script. (Section 19.2 + Section 21 #3.)

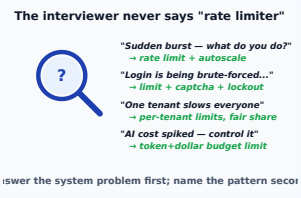
Q11. How would you design a rate limiter for an LLM API?

Two-axis limit: requests/min AND tokens/min (since cost is token-driven). Use a token bucket per user, where each request deducts the actual token count after completion (or estimates upfront for input). Add a daily $-cost cap. (Section 13, AI engineer role; Section 19.)

Q12. How do you handle bursts gracefully?

Token bucket with capacity >> rate (e.g., rate=10/s, capacity=60). Allows a 60-request burst, then sustained 10/s. For clients hitting limits: return 429 with `Retry-After` + jitter so retries don't thunder.

29 Interview questions — indirect



🔍 REAL-WORLD METAPHOR

Reading between the lines

Indirect questions are the real test. Spot the rate-limiting shape inside scenarios about bursts, brute force, noisy tenants, or runaway AI cost — then propose the full toolkit, not just one pattern.

The interviewer never says "rate limit". You're expected to recognize the pattern.

Q1. How would you protect a login API from sudden bursts of failed attempts?

This is rate limiting + brute-force protection. Per-account limit (e.g., 5 fails / 15 min triggers CAPTCHA, 10 / hour triggers temporary lockout). Per-IP limit to catch credential stuffing across many accounts. Plus exponential backoff on response time. Combine with anomaly detection.

Q2. Our autoscaler keeps adding instances at 3am, costing us $$. How do we stop this?

Investigate first — but if it's bots scraping, add IP/UA-based rate limits at the edge so traffic doesn't reach the autoscaling pool. Set a max-instance ceiling. Add per-IP limits at CDN. (Layer-7 limiter at edge stops the load.)

Q3. Design an SMS-OTP system that doesn't bankrupt us if abused.

Per-phone limit (3/5min, 10/day). Per-IP limit. Per-account limit. Global circuit breaker if total volume exceeds 10× baseline. CAPTCHA after 2 failures. (Section 3 in essence.)

Q4. How do you ensure free-tier users can't degrade service for paid users?

Per-tier rate limits + bulkheading. Free tier: 100/day shared pool. Paid tier: dedicated rate per user. At the gateway, route different tiers to different worker pools so a free-tier surge can't starve paid traffic.

Q5. We're calling Stripe and getting 429s ourselves. How do we handle that?

Two angles: (1) implement an outbound rate limiter so we never exceed Stripe's documented limits; (2) on receiving 429, exponential backoff with jitter, queue the request, retry per Stripe's `Retry-After`. Use a token bucket on our side aligned to Stripe's bucket.

Q6. A noisy neighbor on our shared cluster is killing query performance.

Per-tenant query limits at the application layer (e.g., max queries/sec, max concurrent queries, max execution time). Plus DB-level connection pools per tenant (bulkhead). Plus circuit breaker on the offending tenant to fail fast.

Q7. Our API responses are slow during peak hours. We can't add capacity. What do we do?

Several tools, layered: (1) caching for read-heavy endpoints; (2) rate limiting to bound concurrent demand; (3) priority queue (paid users first); (4) load shedding when overload detected; (5) async processing for non-urgent ops. Rate limiting alone won't solve speed — but it will keep p99 sane.

Q8. How would you prevent a single user from running expensive AI queries that cost us $50 each, 100 times in 10 minutes?

Token bucket per user with low refill rate; cost-weighted requests (one expensive op = many tokens deducted). Per-user $/day budget cap. Hard ceiling regardless of plan. Show the user how much they've used so they self-throttle.

Q9. After a deploy, downstream service B is overloaded by service A's retries. How do you mitigate?

Two parts: (a) add an outbound rate limit / concurrency limit on A→B calls; (b) circuit breaker on A so it stops hammering B during failure windows; (c) exponential backoff with jitter on retries. Plus bulkhead between A's other downstreams so the issue doesn't spread.

Q10. Design a fair queueing system for an inference service shared by 1000 customers.

> Per-customer rate limit + per-customer concurrency cap. Use weighted fair queueing — round-robin across customers, weighted by plan tier. Scoring queue items so no customer starves. Combine token bucket (request rate) with concurrency bulkheads (parallel work). For burst absorption: priority queue with deadline.

30 Final cheat-sheet

Decision table — pick your algorithm in under 30 seconds

If you need...	Use this	Where
The simplest possible thing for an MVP	Fixed Window	App middleware
Exact counts, security-critical, low volume	Sliding Window Log	App middleware + Redis sorted set
General-purpose, accurate, O(1) memory	Sliding Window Counter	API Gateway (Kong, Envoy)
Default for most public APIs	Token Bucket	Gateway or app, Redis-backed
Smooth output, fragile downstream	Leaky Bucket	Outbound clients, queue consumers
Massive scale, edge-cached	Sliding Window Counter at edge	Cloudflare, Fastly
Per-tenant fairness in shared cluster	Token Bucket per tenant + bulkhead	App layer + connection pools
LLM / AI / token-cost workloads	Cost-weighted Token Bucket	App layer, with $-cap
Brute-force defense on logins	Sliding Window Log + CAPTCHA	Auth service

The 5-line summary (memorize this)

1. **Rate limiting caps how often a client can do something in a time window** — to protect resources, prevent abuse, and ensure fairness.
2. **Five algorithms exist** — fixed window, sliding window log, sliding window counter, token bucket, leaky bucket. **Default to token bucket.**
3. **Production = Redis + Lua script** for atomic, distributed counters. Always set EXPIRE; always fail-open with alerts; always emit `Retry-After` + jitter.
4. **Limit at multiple layers** — CDN/WAF for volumetric, gateway for API keys/tenants, app for business rules, DB pool for the deepest backstop.
5. **Observe everything** — rejection ratio is the heartbeat metric. Flat near zero = limits too high. Sustained 1–3% = healthy. Sudden spike = attack or surge, investigate now.

Production-ready limit-tuning checklist

- ☐ Measured baseline traffic per endpoint
- ☐ Set limits at 2–3× p99 of legitimate baseline
- ☐ Tested under load — verified no false-positives in normal use
- ☐ Different limits for different tiers (free/pro/enterprise)
- ☐ Composite key (user + endpoint, not just user)
- ☐ `Retry-After` with jitter
- ☐ Standard headers exposed
- ☐ Fail-open on Redis failure + alert
- ☐ Lua script (atomic) — not GET+INCR
- ☐ EXPIRE on all keys
- ☐ Metrics: allowed, rejected, ratio, latency, fail-open count
- ☐ Dashboard with rejection by endpoint, by tier, top rejected keys
- ☐ Alerts on ratio > 5%, fail-open, p99 latency
- ☐ Outbound limits on calls to 3rd-party APIs
- ☐ Health-check endpoints excluded
- ☐ Runbook: how to bump a limit in < 5 min
- ☐ Customer-facing docs: documented limits + retry guidance

> **FINAL THOUGHT**
>
> Rate limiting is the cheapest, highest-leverage piece of infrastructure most teams under-invest in. Add it on day 1 of any public API. Tune it forever. The first time it saves you a 4am page or a $20k AWS bill, you'll wonder how you ever shipped without it.

— End of guide —

YOU MADE IT TO THE END

Thanks for reading.

> *"Rate limiting is the cheapest, highest-leverage piece of infrastructure most teams under-invest in. Add it on day 1 of any public API. Tune it forever."*

THE 5 LINES TO REMEMBER

1. Limit early — before expensive work.
2. Key correctly — user, tenant, route, API key, IP, or composite.
3. Update atomically — Redis + Lua, no read-then-write.
4. Choose the algorithm by fairness, burst, memory, latency.
5. Observe continuously — blocked %, top keys, false positives.

Subscribe to **CodeAsura** for more masterclasses

SYSTEM DESIGN · BACKEND · AI · CLOUD